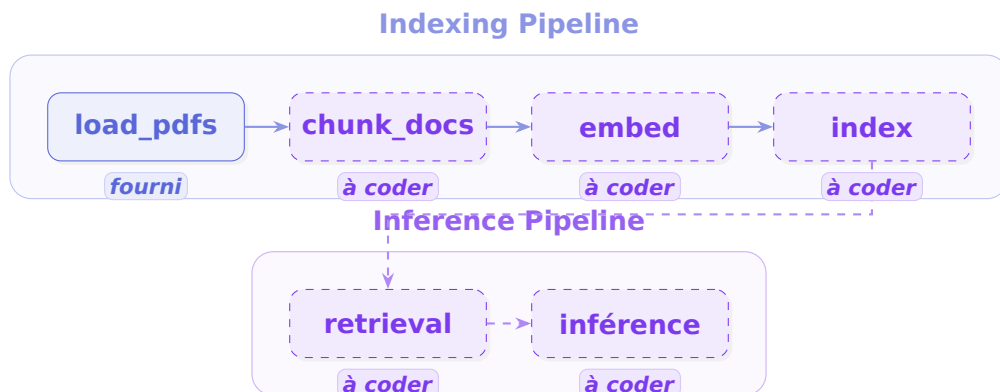


Pipeline RAG avec ZenML

Travaux Pratiques



Niveau	Master 2
Durée	3h
Prérequis	TP RAG avec LangChain (TP2)
Livrables	Code source fonctionnel + rapport

Règle du jeu

Un seul step est fourni en entier (`load_pdfs`). Les autres sont à implémenter par vos soins. Pour chaque step, vous recevez : la **signature**, les **tests unitaires** à passer, et des **indices gradués** (ouvrez-les seulement si vous êtes bloqué). Le TP est réussi quand `pytest tests/` passe au vert.

Table des matières

1	Mise en place (20 min)	2
1.1	Architecture cible	2
1.2	Installation	2
1.3	Fichiers fournis	3
1.3.1	configs/settings.py — Configuration centralisée	3
1.3.2	steps/load_pdfs.py — Le step de référence	5
2	Implémenter les steps d'indexation (60 min)	7
2.1	Step 2 — chunk_documents.py	7
2.2	Step 3 — generate_embeddings.py	9
2.3	Step 4 — index_documents.py	11
3	Assembler le pipeline d'indexation (15 min)	13
4	Retrieval et inférence (45 min)	14
4.1	Fonctions de retrieval	14
4.2	Steps d'inférence	16
4.3	Pipelines d'inférence	18
5	Point d'entrée CLI (20 min)	18
6	Expérimentation et analyse (30 min)	20
7	Évaluation automatique avec RAGAS (30 min)	21
8	Questions de réflexion (dans le rapport)	24
8.1	Livrables attendus	24

Partie 1 Mise en place (20 min)

1.1 Architecture cible

Voici le projet complet que vous allez construire. Les fichiers en **violet** sont à implémenter.

```
rag-zenml-tp/
  configs/
    __init__.py
    settings.py          # fourni
  steps/
    __init__.py          # fourni (a compléter en Partie 7)
    load_pdfs.py         # fourni (step de reference)
    chunk_documents.py   # A IMPLEMENTER
    generate_embeddings.py # A IMPLEMENTER
    index_documents.py   # A IMPLEMENTER
    rag_inference.py     # A IMPLEMENTER
    evaluate_rag.py      # A IMPLEMENTER (Partie 7)
  pipelines/
    __init__.py          # fourni (a compléter en Partie 7)
    indexing_pipeline.py # A IMPLEMENTER
    inference_pipeline.py # A IMPLEMENTER
    evaluation_pipeline.py # A IMPLEMENTER (Partie 7)
  utils/
    __init__.py
    retrieval.py         # A IMPLEMENTER
  tests/
    __init__.py
    test_steps.py        # fourni (tests a passer)
    test_retrieval.py    # fourni (tests a passer)
    test_pipelines.py    # fourni (tests a passer)
    test_evaluation.py    # fourni (tests a passer)
    test_cli.py          # fourni (tests a passer)
  run.py                # A IMPLEMENTER
  requirements.txt       # fourni
```

Note sur les __init__.py fournis

Les fichiers steps/__init__.py et pipelines/__init__.py fournis n'exportent pas encore evaluate_rag ni rag_evaluation_pipeline. Vous les ajouterez vous-mêmes en Partie 7 après avoir implémenté ces fichiers.

1.2 Installation

```
mkdir rag-zenml-tp && cd rag-zenml-tp
mkdir -p configs steps pipelines utils tests pdfs

pip install -r requirements.txt
zenml init
export GROQ_API_KEY="votre_cle"
```

requirements.txt :

```
zenml[server]>=0.60.0
pymupdf>=1.24.0
```

```

chromadb>=0.5.0
sentence-transformers>=3.0.0
langchain-text-splitters>=0.2.0
langchain-groq>=0.1.0
langchain-huggingface>=0.0.3
rank-bm25>=0.2.2
groq>=0.9.0
numpy>=1.26.0
python-dotenv>=1.0.0
pytest>=8.0.0
ragas>=0.2.0,<0.3.0
datasets>=2.18.0

```

Note : ragas, langchain-groq, langchain-huggingface et datasets sont utilisés uniquement en Partie 7. Vous pouvez les installer dès le début ou attendre.

Placez vos PDFs (LoRA, QLoRA, Prefix-Tuning, etc.) dans le dossier pdfs/.

1.3 Fichiers fournis

Les fichiers suivants vous sont donnés tels quels. **Lisez-les attentivement**, ils servent de modèle pour ce que vous devrez coder.

1.3.1 configs/settings.py — Configuration centralisée

```

1  """Configuration centralisee du projet RAG ZenML."""
2  import os
3  from dotenv import load_dotenv
4
5  load_dotenv()
6
7  PDF_DIR = os.getenv("PDF_DIR", "./pdfs")
8
9  DEFAULT_CHUNK_SIZE = 800
10 DEFAULT_CHUNK_OVERLAP = 100
11
12 EMBEDDING_MODEL = (
13     "sentence-transformers/"
14     "paraphrase-multilingual-MiniLM-L12-v2"
15 )
16 EMBEDDING_DIM = 384
17
18 CHROMA_COLLECTION = "rag_tp_collection"
19 CHROMA_PERSIST_DIR = "./chroma_db"
20
21 GROQ_API_KEY = os.getenv("GROQ_API_KEY", "")
22 LLM_MODEL = "llama-3.3-70b-versatile"
23 LLM_TEMPERATURE = 0.0
24 LLM_MAX_TOKENS = 1024
25
26 DEFAULT_TOP_K = 5
27 DEFAULT_BM25_WEIGHT = 0.4
28
29 SYSTEM_PROMPT = """Tu es un assistant expert en IA.
30 Tu reponds aux questions en te basant sur le contexte fourni.
31
32 Regles :
33 - Utilise les informations du contexte pour repondre clairement en francais.
34 - Si le contexte contient une reponse partielle, donne ce que tu peux
35   et precise que l'information est incomplete.
36 - Si le contexte ne contient AUCUNE information pertinente, dis-le.
37 - Ne fais JAMAIS reference au "contexte" dans ta reponse.

```

```
38 - Cite des elements precis du texte pour appuyer ta reponse.  
39  
40 Contexte :  
41 {context}"""
```

Réflexion #1 — Avant de coder

Lisez `settings.py` attentivement. **Pourquoi** centraliser la configuration dans un fichier unique plutôt que de mettre les constantes dans chaque step ? Citez deux scénarios concrets où cette architecture vous simplifie la vie.

Écrivez votre réponse dans le rapport.

1.3.2 steps/load_pdfs.py — Le step de référence

Ce step est **le seul fourni en entier**. Étudiez-le : il illustre toutes les conventions que vos steps devront respecter.

```

1  """Step 1 : Chargement des PDFs."""
2  import glob, logging, os
3  from typing import Annotated, List, Tuple
4
5  import fitz # PyMuPDF
6  from zenml import log_artifact_metadata, step
7
8  logger = logging.getLogger(__name__)
9
10
11 @step(enable_cache=True)
12 def load_pdfs(
13     pdf_dir: str,
14 ) -> Annotated[List[Tuple[str, str]], "raw_documents"]:
15     """Charge tous les PDFs d'un dossier.
16
17     Args:
18         pdf_dir: Chemin vers le dossier des PDFs.
19     Returns:
20         Liste de tuples (contenu_texte, nom_fichier).
21     Raises:
22         ValueError: Si aucun PDF trouve.
23     """
24     pdf_files = sorted(
25         glob.glob(os.path.join(pdf_dir, "*.pdf"))
26     )
27     if not pdf_files:
28         raise ValueError(
29             f"Aucun PDF dans '{pdf_dir}'"
30         )
31
32     documents: List[Tuple[str, str]] = []
33     total_pages = 0
34
35     for pdf_path in pdf_files:
36         filename = os.path.basename(pdf_path)
37         doc = fitz.open(pdf_path)
38         pages = [page.get_text() for page in doc]
39         text = "\n".join(pages)
40         documents.append((text, filename))
41         total_pages += len(doc)
42         doc.close()
43         logger.info("Charge: %s (%d pages)",
44                     filename, len(pages))
45
46     log_artifact_metadata(
47         artifact_name="raw_documents",
48         metadata={
49             "num_pdfs": len(documents),
50             "total_pages": total_pages,
51             "total_chars": sum(
52                 len(d[0]) for d in documents
53             ),
54             "pdf_names": [d[1] for d in documents],
55         },
56     )
57     return documents

```

Réflexion #2 — Anatomie d'un step

En étudiant `load_pdfs`, identifiez les **5 conventions** que tout step ZenML doit respecter dans ce projet. Notez-les dans votre rapport — vous en aurez besoin pour coder les suivants.

Indices : décorateur, typing, nommage, logging, métadonnées.

Partie 2 Implémenter les steps d'indexation (60 min)

Vous devez maintenant implémenter les 3 steps restants du pipeline d'indexation. Pour chaque step, vous recevez la **signature exacte** et les **tests** que votre code doit passer. Le niveau d'aide **diminue** à chaque step.

2.1 Step 2 — chunk_documents.py

Fichier : steps/chunk_documents.py

Signature :

```
@step(enable_cache=True)
def chunk_documents(
    documents: List[Tuple[str, str]],
    chunk_size: int = DEFAULT_CHUNK_SIZE,
    chunk_overlap: int = DEFAULT_CHUNK_OVERLAP,
) -> Annotated[List[Dict[str, str]], "chunks"]:
```

Contrat :

- Reçoit la sortie de load_pdfs (liste de tuples)
- Utilise RecursiveCharacterTextSplitter de LangChain
- Retourne une liste de dicts avec les clés "content", "source", "chunk_id"
- Le chunk_id doit être unique : "filename_chunk_i:04d"
- Doit logger les métadonnées : chunk_size, chunk_overlap, total_chunks, avg_chunk_length

```

1 def test_chunk_documents_output_format():
2     """Chaque chunk a les bonnes cles."""
3     docs = [("Texte long. " * 200, "test.pdf")]
4     result = chunk_documents(
5         documents=docs, chunk_size=100,
6         chunk_overlap=20,
7     )
8     assert len(result) > 1
9     for chunk in result:
10         assert "content" in chunk
11         assert "source" in chunk
12         assert "chunk_id" in chunk
13         assert chunk["source"] == "test.pdf"
14         assert len(chunk["content"]) <= 120 # marge
15
16 def test_chunk_documents_overlap():
17     """L'overlap fonctionne : le debut du chunk N+1
18     contient la fin du chunk N."""
19     docs = [("Mot " * 500, "test.pdf")]
20     result = chunk_documents(
21         documents=docs, chunk_size=100,
22         chunk_overlap=30,
23     )
24     if len(result) >= 2:
25         fin = result[0]["content"][-30:]
26         debut = result[1]["content"][:30]
27         # Au moins une partie commune
28         common = set(fin.split()) & set(debut.split())
29         assert len(common) > 0
30
31 def test_chunk_documents_unique_ids():
32     """Tous les chunk_id sont uniques."""
33     docs = [("Texte. " * 200, "a.pdf"),
34             ("Autre. " * 200, "b.pdf")]
35     result = chunk_documents(documents=docs,
36                             chunk_size=50,
37                             chunk_overlap=10)
38     ids = [c["chunk_id"] for c in result]
39     assert len(ids) == len(set(ids))

```

Indice (ouvrez seulement si bloqué 10+ min)

Les séparateurs à utiliser, du plus prioritaire au moins prioritaire : ["\n\n", "\n", ". ", " ", ""]. Importez les defaults depuis `configs.settings`.

2.2 Step 3 — generate_embeddings.py

Fichier : steps/generate_embeddings.py

Signature :

```
@step(enable_cache=True)
def generate_embeddings(
    chunks: List[Dict[str, str]],
    model_name: str = EMBEDDING_MODEL,
) -> Annotated[List[Dict], "chunks_with_embeddings"]:
```

Contrat :

- Reçoit la sortie de chunk_documents
- Charge le modèle avec SentenceTransformer(model_name)
- Encode les textes avec normalize_embeddings=True
- Retourne les mêmes dicts, enrichis d'une clé "embedding" (liste de floats)
- Ne doit **pas** modifier les dicts d'entrée (copie)

```
1 def test_embeddings_added():
2     """Chaque chunk contient un embedding."""
3     chunks = [
4         {"content": "LoRA réduit les parametres",
5          "source": "t.pdf", "chunk_id": "t_0000"},
6         {"content": "QLoRA quantifie en 4 bits",
7          "source": "t.pdf", "chunk_id": "t_0001"},
8     ]
9     result = generate_embeddings(chunks=chunks)
10    assert len(result) == 2
11    for r in result:
12        assert "embedding" in r
13        assert isinstance(r["embedding"], list)
14        assert len(r["embedding"]) == 384
15
16 def test_embeddings_normalized():
17     """Les embeddings sont normalises (norme ~1)."""
18     import numpy as np
19     chunks = [{"content": "Test embedding",
20                "source": "t.pdf",
21                "chunk_id": "t_0000"}]
22     result = generate_embeddings(chunks=chunks)
23     vec = np.array(result[0]["embedding"])
24     norm = np.linalg.norm(vec)
25     assert abs(norm - 1.0) < 0.01
26
27 def test_embeddings_no_mutation():
28     """Les dicts d'entree ne sont pas modifies."""
29     chunks = [{"content": "Test", "source": "t.pdf",
30                "chunk_id": "t_0000"}]
31     original_keys = set(chunks[0].keys())
32     generate_embeddings(chunks=chunks)
33     assert set(chunks[0].keys()) == original_keys
```

Réflexion #3 — Choix de conception

Pourquoi le contrat exige-t-il de **ne pas modifier les dicts d'entrée** ? Quel problème cela pourrait-il causer dans un pipeline ZenML avec cache activé ?

2.3 Step 4 — index_documents.py

Le niveau d'aide diminue. Vous n'avez plus que la signature et les tests.

Fichier : steps/index_documents.py

Signature :

```
@step(enable_cache=False)
def index_documents(
    chunks_with_embeddings: List[Dict],
    collection_name: str = CHROMA_COLLECTION,
    persist_dir: str = CHROMA_PERSIST_DIR,
) -> str:
```

Contrat :

- Crée (ou recrée) une collection ChromaDB persistée sur disque
- Insère les chunks avec embeddings et métadonnées de source
- Retourne le nom de la collection
- Doit gérer l'insertion par **batch** (ChromaDB a une limite par appel)

```

1 import chromadb, tempfile
2
3 def test_index_creates_collection():
4     """La collection est creee et contient
5     les bons vecteurs."""
6     with tempfile.TemporaryDirectory() as tmpdir:
7         chunks = [
8             {"content": "Texte A", "source": "a.pdf",
9              "chunk_id": "a_0000",
10             "embedding": [0.1] * 384},
11             {"content": "Texte B", "source": "b.pdf",
12              "chunk_id": "b_0000",
13              "embedding": [0.2] * 384},
14         ]
15         name = index_documents(
16             chunks_with_embeddings=chunks,
17             collection_name="test_col",
18             persist_dir=tmpdir,
19         )
20         assert name == "test_col"
21         client = chromadb.PersistentClient(
22             path=tmpdir
23         )
24         col = client.get_collection("test_col")
25         assert col.count() == 2
26
27 def test_index_idempotent():
28     """Relancer l'indexation repart de zero
29     (pas de doublons)."""
30     with tempfile.TemporaryDirectory() as tmpdir:
31         chunks = [
32             {"content": "X", "source": "x.pdf",
33              "chunk_id": "x_0",
34              "embedding": [0.5] * 384},
35         ]
36         index_documents(
37             chunks_with_embeddings=chunks,
38             collection_name="test_idem",
39             persist_dir=tmpdir,
40         )
41         index_documents(
42             chunks_with_embeddings=chunks,
43             collection_name="test_idem",
44             persist_dir=tmpdir,
45         )
46         client = chromadb.PersistentClient(
47             path=tmpdir
48         )
49         col = client.get_collection("test_idem")
50         assert col.count() == 1

```

Réflexion #4 — Cache désactivé

Ce step a `enable_cache=False`. Les trois précédents ont `True`. Expliquez **pour-quoi** en une phrase. Que se passerait-il si on mettait `True` ici ?

Partie 3 Assembler le pipeline d'indexation (15 min)

Fichier : pipelines/indexing_pipeline.py

Enchaînez les 4 steps dans un pipeline ZenML nommé "rag_indexing_pipeline". Le pipeline doit accepter 3 paramètres : pdf_dir, chunk_size, chunk_overlap.

Signature :

```
@pipeline(name="rag_indexing_pipeline",
          enable_cache=True)
def rag_indexing_pipeline(
    pdf_dir: str = PDF_DIR,
    chunk_size: int = DEFAULT_CHUNK_SIZE,
    chunk_overlap: int = DEFAULT_CHUNK_OVERLAP,
) -> str:
```

```
1 def test_indexing_pipeline_runs(tmp_pdf_dir):
2     """Le pipeline s'exécute sans erreur
3     et produit une collection."""
4     result = rag_indexing_pipeline(
5         pdf_dir=tmp_pdf_dir,
6         chunk_size=200,
7         chunk_overlap=20,
8     )
9     # Vérifier que la collection existe
10    client = chromadb.PersistentClient(
11        path=CHROMA_PERSIST_DIR
12    )
13    col = client.get_collection(CHROMA_COLLECTION)
14    assert col.count() > 0
```

Vérification intermédiaire

À ce stade, lancez :

```
pytest tests/test_steps.py tests/test_pipelines.py -v
```

Tous les tests des steps et du pipeline d'indexation doivent passer. Si ce n'est pas le cas, corrigez avant de continuer.

Réflexion #5 — Le cache en action

Exécutez le pipeline deux fois de suite avec les mêmes paramètres.

```
rag_indexing_pipeline(pdf_dir="./pdfs")
rag_indexing_pipeline(pdf_dir="./pdfs")
```

Questions :

1. Quels steps sont "cached" au deuxième run ?
2. Lequel est toujours re-exécuté ? Pourquoi ?
3. Relancez avec chunk_size=400. Le step load_pdfs est-il recalculé ?

Partie 4 Retrieval et inférence (45 min)

Ici, **aucun code n'est fourni**. Vous avez les signatures, les tests, et votre connaissance du TP2 (RAG avec LangChain).

4.1 Fonctions de retrieval

Fichier : `utils/retrieval.py`

```
def dense_retrieve(
    query: str,
    top_k: int = DEFAULT_TOP_K,
    collection_name: str = CHROMA_COLLECTION,
    persist_dir: str = CHROMA_PERSIST_DIR,
) -> List[Dict]:
    """Retourne les top_k chunks les plus proches.

    Chaque dict contient : 'content', 'source',
    'distance'.
    """
```

```
def hybrid_retrieve(
    query: str,
    top_k: int = DEFAULT_TOP_K,
    bm25_weight: float = DEFAULT_BM25_WEIGHT,
    collection_name: str = CHROMA_COLLECTION,
    persist_dir: str = CHROMA_PERSIST_DIR,
) -> List[Dict]:
    """Combine BM25 et Dense avec fusion par rang.

    Chaque dict contient : 'content', 'source',
    'score'.
    """
```

Algorithme de fusion à implémenter :

$$\text{score}(d) = \sum_{r \in \{\text{bm25}, \text{dense}\}} \frac{w_r}{\text{rang}_r(d) + 1}$$

où $w_{\text{bm25}} = \text{bm25_weight}$ et $w_{\text{dense}} = 1 - \text{bm25_weight}$.

```
1 def test_dense_retrieve_format(indexed_collection):
2     """dense_retrieve retourne le bon format."""
3     results = dense_retrieve(
4         "test query", top_k=3,
5         **indexed_collection,
6     )
7     assert len(results) <= 3
8     for r in results:
9         assert "content" in r
10        assert "source" in r
11        assert "distance" in r
12
13 def test_hybrid_retrieve_format(indexed_collection):
14     """hybrid_retrieve retourne le bon format."""
15     results = hybrid_retrieve(
16         "test query", top_k=3,
17         bm25_weight=0.4,
18         **indexed_collection,
19     )
20     assert len(results) <= 3
21     for r in results:
22         assert "content" in r
23         assert "source" in r
24         assert "score" in r
25
26 def test_hybrid_weights_matter(indexed_collection):
27     """Changer bm25_weight change le classement."""
28     r1 = hybrid_retrieve(
29         "LoRA rank", bm25_weight=0.9,
30         **indexed_collection,
31     )
32     r2 = hybrid_retrieve(
33         "LoRA rank", bm25_weight=0.1,
34         **indexed_collection,
35     )
36     # Au moins un resultat different en position 1
37     if len(r1) > 0 and len(r2) > 0:
38         # On ne peut pas garantir un ordre different
39         # mais les scores doivent etre differents
40         assert r1[0]["score"] != r2[0]["score"]
```

4.2 Steps d'inférence

Fichier : steps/rag_inference.py

Implémentez **deux steps** :

```
@step
def rag_inference(
    question: str,
    collection_name: str = CHROMA_COLLECTION,
    persist_dir: str = CHROMA_PERSIST_DIR,
    top_k: int = DEFAULT_TOP_K,
) -> Dict[str, str]:
    """Inference RAG avec retrieval dense.
    Retourne : question, answer, sources,
               retriever, num_chunks.
    """

@step
def rag_hybrid_inference(
    question: str,
    collection_name: str = CHROMA_COLLECTION,
    persist_dir: str = CHROMA_PERSIST_DIR,
    top_k: int = DEFAULT_TOP_K,
    bm25_weight: float = DEFAULT_BM25_WEIGHT,
) -> Dict[str, str]:
    """Inference RAG avec retrieval hybride.
    Retourne : question, answer, sources,
               retriever, bm25_weight, num_chunks.
    """
```

Contrat :

- Utiliser vos fonctions de utils/retrieval.py
- Construire le contexte en joignant les chunks par "\n\n---\n\n"
- Appeler le LLM Groq avec le SYSTEM_PROMPT de settings.py
- Factoriser l'appel LLM dans une fonction privée _call_llm(question, context)

```
1 def test_rag_inference_output_keys():
2     """Le dict de sortie contient les bonnes cles."""
3     result = rag_inference(question="Test ?")
4     expected_keys = {"question", "answer", "sources",
5                      "retriever", "num_chunks"}
6     assert expected_keys <= set(result.keys())
7     assert result["retriever"] == "dense"
8
9 def test_rag_hybrid_output_keys():
10    """Le dict hybride contient bm25_weight."""
11    result = rag_hybrid_inference(
12        question="Test ?", bm25_weight=0.5,
13    )
14    assert "bm25_weight" in result
15    assert result["retriever"] == "hybrid_bm25_dense"
```

Réflexion #6 — Conception logicielle

Pourquoi le contrat vous demande-t-il de factoriser l'appel LLM dans `_call_llm`? Nommez deux avantages de ce choix par rapport à dupliquer le code dans chaque step.

4.3 Pipelines d'inférence

Fichier : pipelines/inference_pipeline.py

Écrivez deux pipelines :

- rag_dense_inference_pipeline(question) qui appelle rag_inference
- rag_hybrid_inference_pipeline(question, bm25_weight) qui appelle rag_hybrid_inference

Vous savez déjà comment écrire un pipeline — c'est le plus simple de tout le TP.

Partie 5 Point d'entrée CLI (20 min)

Fichier : run.py

Créez un CLI avec argparse qui expose **trois sous-commandes** :

```
python run.py index --pdf-dir ./pdfs \
                  --chunk-size 800 \
                  --chunk-overlap 100

python run.py query --question "Comment LoRA ?" \
                  [--hybrid] \
                  [--bm25-weight 0.4]

python run.py history [--limit 10]
```

Contrat :

- index : appelle rag_indexing_pipeline avec les paramètres
- query : appelle le pipeline dense ou hybride selon --hybrid, puis affiche la réponse en récupérant l'artefact de sortie via Client()
- history : liste les N derniers runs de chaque pipeline avec nom, status, date
- Sans argument : affiche l'aide (parser.print_help())

```
1 import subprocess
2
3 def test_cli_help():
4     """run.py sans argument affiche l'aide."""
5     r = subprocess.run(
6         ["python", "run.py"],
7         capture_output=True, text=True,
8     )
9     assert "index" in r.stdout or "index" in r.stderr
10    assert "query" in r.stdout or "query" in r.stderr
```

Indice pour query

Pour récupérer le résultat d'un pipeline après exécution :

```
from zenml.client import Client
client = Client()
run = client.get_pipeline("nom_pipeline").last_run
output = run.steps["nom_step"].output.load()
```

Partie 6 Expérimentation et analyse (30 min)

Votre système est fonctionnel. Il est temps de l'évaluer.

Exécutez le pipeline avec trois configurations et posez la même question à chaque fois :

```
python run.py index --chunk-size 400 --chunk-overlap 50
python run.py query --question \
    "Quelle difference entre LoRA et QLoRA ?"

python run.py index --chunk-size 800 --chunk-overlap 100
python run.py query --question \
    "Quelle difference entre LoRA et QLoRA ?"

python run.py index --chunk-size 1500 --chunk-overlap 200
python run.py query --question \
    "Quelle difference entre LoRA et QLoRA ?"
```

Rapport : Remplissez ce tableau et justifiez votre classement de qualité.

chunk_size	overlap	Nb chunks	Qualité (1-5)	Justification
400	50			
800	100			
1500	200			

Testez les deux retrievers sur trois types de questions et analysez les différences.

Type de question	Dense	Hybrid	Analyse
Termes techniques exacts (ex : "Quel rank r=8 dans LoRA?")	ok/ko	ok/ko	
Question paraphrasée (ex : "Comment réduire les params?")	ok/ko	ok/ko	
Question comparative (ex : "Compare LoRA et QLoRA")	ok/ko	ok/ko	

Rapport : Sur quel type de question le retriever hybride apporte-t-il le plus de valeur? Expliquez **mécaniquement** pourquoi (en termes de BM25 vs similarité cosinus).

Partie 7 Évaluation automatique avec RAGAS (30 min)

Jusqu'ici, vous avez évalué la qualité du RAG **manuellement** ("la réponse est-elle bonne?"). En production, il faut des métriques automatiques. RAGAS utilise un LLM-juge pour mesurer trois aspects :

Métrique	Question posée	Détecte
Faithfulness	La réponse est-elle fidèle aux chunks ?	Hallucinations
Answer Relevancy	La réponse répond-elle à la question ?	Hors-sujet
Context Precision	Les chunks récupérés sont-ils utiles ?	Bruit retrieval

Fichier : steps/evaluate_rag.py

```
@step(enable_cache=False)
def evaluate_rag(
    collection_name: str = CHROMA_COLLECTION,
    persist_dir: str = CHROMA_PERSIST_DIR,
    eval_questions: List[str] = DEFAULT_EVAL_QUESTIONS,
    top_k: int = DEFAULT_TOP_K,
) -> Annotated[
    Dict[str, object], "evaluation_results"
]:
```

Contrat :

- Pour chaque question : (1) récupérer les chunks via `dense_retrieve`, (2) générer une réponse via le LLM, (3) stocker question + réponse + contextes
- Construire un Dataset HuggingFace avec les clés `user_input`, `response`, `retrieved_contexts`
- Évaluer avec les métriques RAGAS :
 - `Faithfulness(llm=ragas_llm)`
 - `ResponseRelevancy(llm=ragas_llm, embeddings=ragas_emb)`
 - `LLMContextPrecisionWithoutReference(llm=ragas_llm)`
- Retourner un dict avec : `avg_faithfulness`, `avg_answer_relevancy`, `avg_context_precision`, `num_questions`, `per_question` (liste de dicts), `low_faithfulness_questions`, `low_precision_questions`

Indice — Configurer RAGAS avec Groq

RAGAS a besoin d'un LLM-juge. Utilisez le wrapper LangChain :

```
from ragas.llms import LangchainLLMWrapper
from ragas.embeddings import LangchainEmbeddingsWrapper
from langchain_groq import ChatGroq
from langchain_huggingface import HuggingFaceEmbeddings

ragas_llm = LangchainLLMWrapper(
    ChatGroq(model=LLM_MODEL, temperature=0,
             api_key=api_key)
)
ragas_emb = LangchainEmbeddingsWrapper(
    HuggingFaceEmbeddings(
        model_name=EMBEDDING_MODEL,
        model_kwargs={"device": "cpu"},
    )
)
```

```
1 def test_default_questions_exist():
2     """Les questions par défaut sont définies."""
3     from steps.evaluate_rag import DEFAULT_EVAL QUESTIONS
4     assert isinstance(DEFAULT_EVAL QUESTIONS, list)
5     assert len(DEFAULT_EVAL QUESTIONS) >= 3
6
7 def test_step_is_decorated():
8     """evaluate_rag est un step ZenML."""
9     from steps.evaluate_rag import evaluate_rag
10    assert hasattr(evaluate_rag, "entrypoint")
```

Fichiers :

- pipelines/evaluation_pipeline.py — encapsuler evaluate_rag dans un pipeline nommé "rag_evaluation_pipeline"
- run.py — ajouter la sous-commande eval :

```
python run.py eval [--top-k 5]
```

La commande doit afficher les scores moyens et identifier les questions problématiques.

Lancez l'évaluation et remplissez ce tableau :

Question	Faith.	Relev.	Diagnostic
Comment LoRA réduit...			
Différence LoRA / QLoRA			
Prefix-Tuning...			
Avantages QLoRA...			
Rang r dans LoRA...			

Rapport :

1. Quelle métrique est la plus faible ? Qu'est-ce que cela indique ?
2. Si Faithfulness est faible : est-ce un problème de retrieval ou de prompt ?
3. Si Context Precision est faible : quel paramètre ajuster ?
4. Modifiez un paramètre (chunk_size, k, ou prompt), relancez l'évaluation, et montrez l'amélioration.

Partie 8 Questions de réflexion (dans le rapport)

En plus des réflexions #1 à #6 intégrées au fil du TP, répondez à ces questions dans votre rapport :

1. **Débogage** : Votre RAG invente la valeur " $r = 16$ " alors que le paper dit $r = 8$. Décrivez la **méthodologie exacte** que vous suivriez pour trouver l'origine du bug, en utilisant les artefacts ZenML.
2. **Production** : Vous déployez ce RAG dans une entreprise avec 10 000 PDFs et 50 utilisateurs simultanés. Identifiez **trois** composants que vous changeriez et justifiez chaque choix.
3. **RAGAS en production** : Comment automatiseriez-vous l'évaluation RAGAS pour qu'elle se déclenche **à chaque mise à jour** du corpus de PDFs ? Décrivez le pipeline complet (indexation → évaluation → alerte).

8.1 Livrables attendus

Livrable	Critère de validation
Code source	pytest tests/ -v passe au vert (tous les tests)
Rapport	Réponses aux réflexions #1-#6 + questions 1-3
Captures	Output de python run.py history et des expériences
Tableaux	Résultats complétés des exercices 11 et 12 avec justifications

Récapitulatif — comment vérifier votre travail

```
# Tous les tests doivent passer
pytest tests/ -v

# Le pipeline d'indexation doit tourner
python run.py index --pdf-dir ./pdfs

# Les requetes doivent retourner des reponses
python run.py query --question "Comment LoRA ?"
python run.py query --question "Comment LoRA ?" --hybrid

# L'historique doit lister vos runs
python run.py history
```